CUDA lab 2

Dries Van de Velde, Behzad Iqbal

February 2024

1 Introduction

The goal of this lab is to compare the following techniques for retrieving the maximum value of an array. CPU, GPU atomic operation, GPU synchronisation.

2 Atomic operation

This is by far the easiest implementation, however there is a lot of thread locking with this approach which means that the GPU spends a large portion of its time waiting. This could theoretically be solved using reduction which will be explained in the next step.

3 Reduction

When implementing the reduction algorithm we ran into the problem that the maximum allowed block size is 1024. Since there is no method to do synchronisation on block level, we opted for an approach in which the GPU function gets called multiple times in which the array gets divided into parts of 1024.
This does not give the expected timing results for large arrays. We expect this is because of the repeated function calls and their GPU stream with their corresponding overhead.
A solution to this problem could be the usage of shared memory for faster memory read/write times.
The reduction algorithm also doesn't meet the expected timing results, for the small array sizes for which there are not many serialised GPU function calls needed the timings aren't that fast compared to the atomic operation and even the CPU alternative.

4 How would you tackle a 2D array?

When calculating 2D arrays, 2D blocks can be used. This means splitting up the block sizes dynamically so the parallelisation can be used to its full extend.

5 Plot

In the plot the array size and the execution time are both logarithmically plotted. The benefit of using parallelisation by utilising the GPU becomes clear. Especially the Atomic operation has big time advantages once the array starts getting to a reasonably large size.

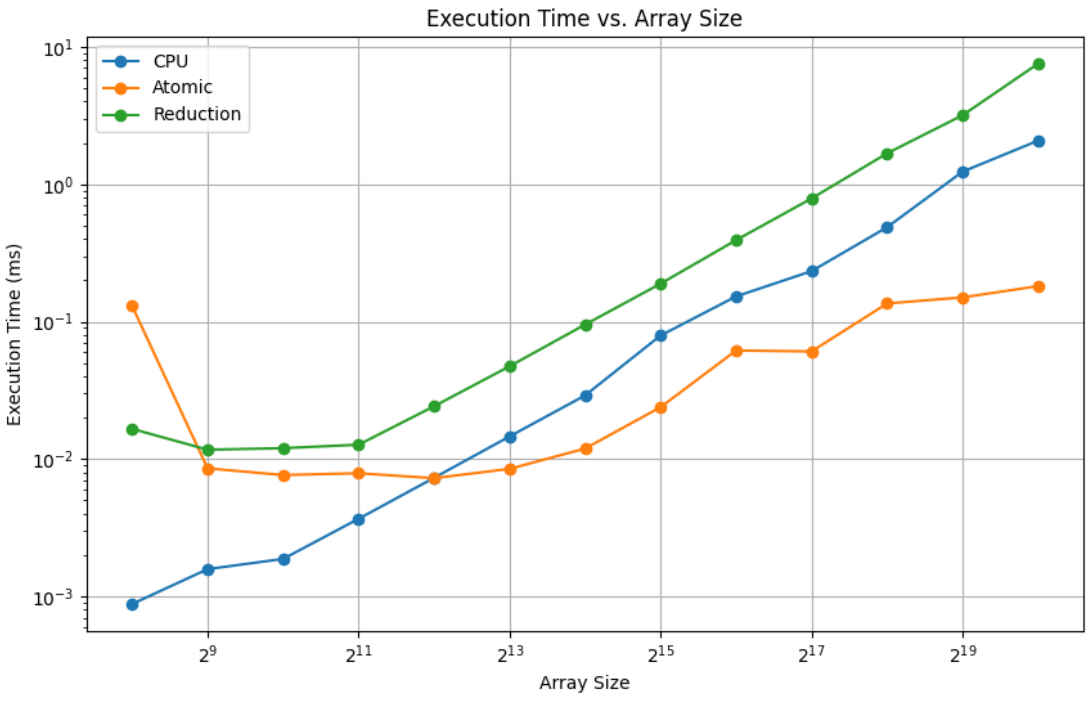


Figure 1: Execution times CPU, Atomic operation, Reduction